

Rock Paper Scissors

Project & Code Explanation

Java 17+ · Spring Boot 3 · Vue 3
Object-oriented design walkthrough

Contents

1. Project overview . . . 3
    1.1 Technology stack . . . 3
    1.2 Features . . . 3
2. Architecture . . . 4
    2.1 Folder structure . . . 5
3. Object-oriented concepts used . . . 6
    3.1 Class diagram . . . 6
4. Module-by-module explanation . . . 7
    4.1 Application entry point . . . 7
    4.2 model package – the rules and game state . . . 7
    4.3 player package – who plays . . . 8
    4.4 strategy package – the computer's brain . . . 8
    4.5 dto package – data sent over HTTP . . . 9
    4.6 service package – business logic . . . 9
    4.7 controller package – the REST API . . . 10
    4.8 Frontend – index.html and style.css . . . 10
    4.9 Tests and build file . . . 10
5. What happens when you click “Rock” . . . 11
6. How to run the project . . . 11
7. Limitations and ideas for improvement . . . 11

1. Project overview

This project is a browser-based **Rock Paper Scissors** game. You play against the computer, which can be **Easy** (plays randomly) or **Hard** (learns which move you play most and counters it). The game keeps score and shows the history of every round.

The project is split into two parts:

- **Backend (Java + Spring Boot):** all the game rules, players, computer AI and scoring. It is written with object-oriented principles and exposes a small REST API.
- **Frontend (Vue 3):** a single web page that shows buttons, the score and history. It talks to the backend using HTTP requests and JSON.

1.1 Technology stack

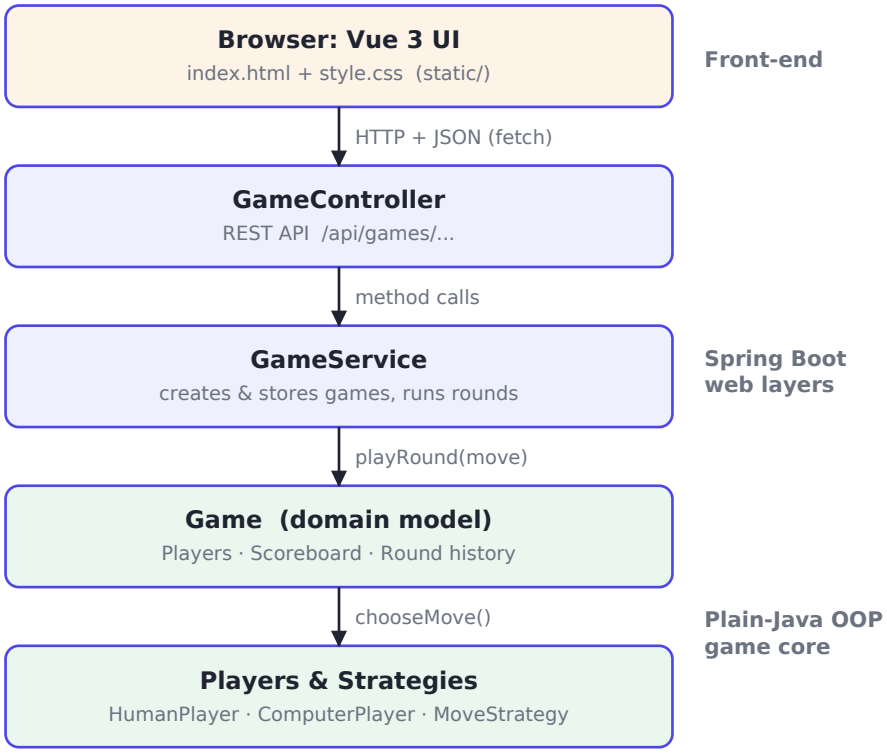
Part	Technology	Why
Language	Java 17+	Records, switch expressions, enums – modern, concise OOP.
Backend framework	Spring Boot 3.3 (spring-boot-starter-web)	Embedded web server, REST controllers and dependency injection with almost no setup.
Build tool	Maven (pom.xml)	Downloads dependencies, compiles, tests and packages the app.
Frontend	Vue 3 (loaded from CDN)	Reactive UI with no Node.js/npm build step.
Testing	JUnit 5	Checks game rules, scoring and the AI strategy.

1.2 Features

- Enter your name and choose a difficulty.
- Click Rock, Paper or Scissors to play a round.
- See both moves, the result (win / lose / draw) and the running score.
- Full round-history table (newest first).
- Reset the score or go back and change settings.

2. Architecture

The application follows a classic **layered architecture**. Each layer has one job and only talks to the layer directly below it. This makes the code easier to understand, test and change.



Layer	Package / file	Responsibility
Presentation	static/index.html, style.css	Shows the game and sends the player's actions to the API.
Controller	controller	Receives HTTP requests, converts JSON, returns responses. No game logic.
Service	service	Business logic: create/find/reset games, play a round safely.
DTO	dto	Simple data objects that define the JSON shape of requests and responses.
Domain model	model, player, strategy	Pure Java OOP: rules, players, AI, score. No Spring code at all.

Notice that the game core (`model`, `player`, `strategy`) does not depend on Spring. You could reuse exactly the same classes in a console game or a JavaFX desktop app.

2.1 Folder structure

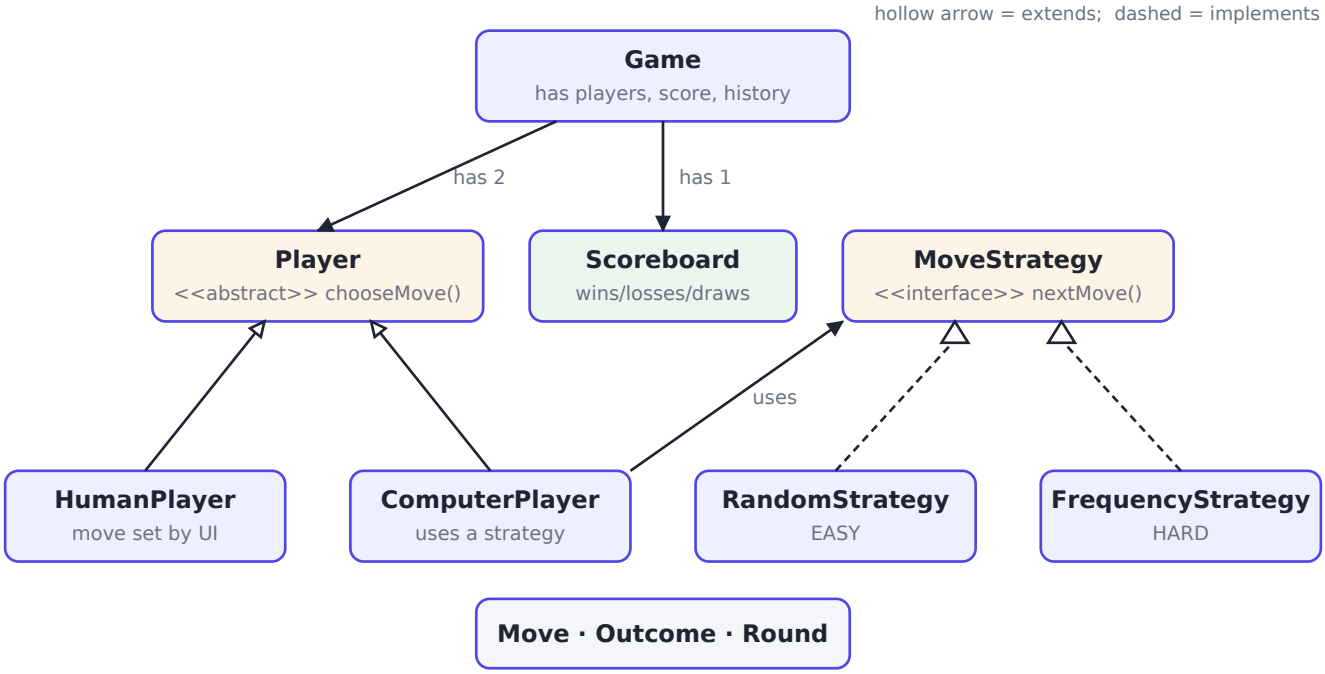
```
rock-paper-scissors/
├── pom.xml                              Maven build file
└── src/
    ├── main/java/com/example/rps/
    │   ├── RockPaperScissorsApplication.java   entry point
    │   ├── model/        Move, Outcome, Round, Scoreboard, Game
    │   ├── player/       Player, HumanPlayer, ComputerPlayer
    │   ├── strategy/     MoveStrategy, RandomStrategy, FrequencyStrategy, Difficulty
    │   ├── service/      GameService, GameNotFoundException
    │   ├── dto/          NewGameRequest, PlayRequest, GameStateResponse
    │   └── controller/   GameController
    ├── main/resources/static/
    │   ├── index.html    Vue 3 user interface
    │   └── style.css     styling
    └── test/java/com/example/rps/
        └── GameLogicTest.java   JUnit tests
```

3. Object-oriented concepts used

The table below maps each OOP principle to the exact place it is used in the code.

Concept	Where	How it is used
Abstraction	Player (abstract class), MoveStrategy (interface)	The game only needs to know that a player can chooseMove(). The details are hidden in subclasses.
Encapsulation	Scoreboard, Game, HumanPlayer	Fields are private. The score can only change through record(); history is returned as an unmodifiable list.
Inheritance	HumanPlayer, ComputerPlayer extend Player	Both reuse the name field and getName() from Player and override chooseMove().
Polymorphism	Player.chooseMove(), MoveStrategy.nextMove()	The same call behaves differently depending on the real object: human vs computer, random vs learning AI.
Interface + default methods	MoveStrategy	observe() and reset() have default empty bodies, so simple strategies don't need to implement them.
Composition	Game, ComputerPlayer	A Game has players, a scoreboard and rounds. A ComputerPlayer has a strategy.
Strategy pattern	ComputerPlayer + MoveStrategy	The AI behaviour is swappable without changing ComputerPlayer (open/closed principle).
Factory (via enum)	Difficulty	Each difficulty constant knows how to create its strategy: EASY → RandomStrategy, HARD → FrequencyStrategy.
Immutability	Round (record), DTO records, final fields	Objects that should never change after creation are records or have final fields.
Rich enums	Move, Outcome	Enums carry data (emoji) and behaviour (beats(), decide()) instead of scattered if/else.

3.1 Class diagram



4. Module-by-module explanation

This section walks through every package and file in the order the game uses them: from the basic rules up to the web API.

4.1 Application entry point

RockPaperScissorsApplication.java – contains `main()`. The `@SpringBootApplication` annotation turns on auto-configuration and component scanning, so Spring finds `GameService` and `GameController` automatically and starts an embedded Tomcat server on port 8080. It also serves everything in `resources/static`, which is how the Vue page is delivered.

```
@SpringBootApplication
public class RockPaperScissorsApplication {
    public static void main(String[] args) {
        SpringApplication.run(RockPaperScissorsApplication.class, args);
    }
}
```

4.2 model package – the rules and game state

Move.java (enum) – the three moves ROCK, PAPER and SCISSORS. Each constant stores an emoji and knows the game rules itself:

- `beatsMove()` – the move this one defeats (Rock → Scissors, Paper → Rock, Scissors → Paper).
- `losesTo()` – the move that defeats this one. Used by the Hard AI to counter you.
- `beats(other)` – true if this move wins against other.

```
public Move beatsMove() {
    return switch (this) {
        case ROCK -> SCISSORS;
        case PAPER -> ROCK;
        case SCISSORS -> PAPER;
    };
}
public boolean beats(Move other) { return beatsMove() == other; }
```

Outcome.java (enum) – WIN, LOSE or DRAW from the human's point of view. The static method `decide(player, computer)` holds the entire winner logic in three lines:

```
public static Outcome decide(Move player, Move computer) {
    if (player == computer) return DRAW;
    return player.beats(computer) ? WIN : LOSE;
}
```

Round.java (record) – an immutable snapshot of one round: round number, player move, computer move and outcome. A Java record automatically generates the constructor, getters, `equals()`, `hashCode()` and `toString()`.

Scoreboard.java – keeps wins, losses and draws in **private** fields. The only way to change them is `record(outcome)` or `reset()`, so the score can never get into an invalid state (encapsulation). `getTotal()` is calculated, not stored.

Game.java – the heart of the game. It is composed of a HumanPlayer, a ComputerPlayer, a Scoreboard and a list of Rounds. `playRound(move)` runs one round:

```
public Round playRound(Move humanMove) {
    human.setNextMove(humanMove);                    // 1. give the human their move

    Move playerMove = human.chooseMove();            // 2. ask both players (polymorphism)
    Move computerMove = computer.chooseMove();
    Outcome outcome = Outcome.decide(playerMove, computerMove);   // 3. decide

    Round round = new Round(history.size() + 1, playerMove, computerMove, outcome);
```

```
    history.add(round);                              // 4. remember it
    scoreboard.record(outcome);                      // 5. update the score
    computer.observeOpponent(playerMove);            // 6. let the AI learn
    return round;
}
```

`getHistory()` returns `Collections.unmodifiableList(history)` so code outside the class can read the history but not modify it. `reset()` clears the history and score and makes the AI forget what it learned.

4.3 player package – who plays

Player.java (abstract class) – the common parent of every player. It stores the player's `name` (private and final) and declares the abstract method `chooseMove()`. Because it is abstract, you cannot create a plain `Player`; you must use a concrete subclass.

```
public abstract class Player {
    private final String name;
    protected Player(String name) { this.name = name; }
    public String getName() { return name; }
    public abstract Move chooseMove();
}
```

HumanPlayer.java – extends Player. The human's move comes from the button clicked in the browser, so it is stored with `setNextMove()` and simply returned by `chooseMove()`. If no move was set it throws `IllegalStateException` (defensive programming).

ComputerPlayer.java – extends Player but does not decide moves itself. It holds a `MoveStrategy` and delegates to it. This is the **Strategy design pattern**: the same ComputerPlayer class can be easy or hard depending on which strategy object it is given.

```
public class ComputerPlayer extends Player {
    private final MoveStrategy strategy;
    @Override
    public Move chooseMove() { return strategy.nextMove(); }
    public void observeOpponent(Move m) { strategy.observe(m); }
    public void forget() { strategy.reset(); }
}
```

4.4 strategy package – the computer's brain

MoveStrategy.java (interface) – the contract for any AI: `nextMove()` and `getName()` must be implemented. `observe()` and `reset()` are default methods with empty bodies, so a strategy that doesn't learn can ignore them.

RandomStrategy.java – the EASY opponent. Picks a random element of `Move.values()`. It has a second constructor that accepts a `Random` object, which makes it possible to inject a seeded Random in tests.

FrequencyStrategy.java – the HARD opponent. It keeps an `EnumMap<Move, Integer>` counting how often you played each move. Before each round it finds your most frequent move and plays the move that beats it (`favourite.losesTo()`). With no data yet, it falls back to a RandomStrategy – another example of composition.

```
@Override
public Move nextMove() {
    Move favourite = null;
    int best = 0;
    for (Map.Entry<Move, Integer> e : counts.entrySet()) {
        if (e.getValue() > best) { best = e.getValue(); favourite = e.getKey(); }
    }
    return favourite == null ? fallback.nextMove() : favourite.losesTo();
}
```

```
@Override
public void observe(Move opponentMove) { counts.merge(opponentMove, 1, Integer::sum); }
```

Difficulty.java (enum) – EASY and HARD. Each constant holds a `Supplier<MoveStrategy>` (a factory written as a constructor reference such as `RandomStrategy::new`). `createStrategy()` builds a fresh strategy object for each new game, so two games never share AI memory. Adding a new level only needs a new enum constant.

4.5 dto package – data sent over HTTP

DTOs (Data Transfer Objects) define the exact JSON that travels between browser and server. They are Java records, so they are immutable and Spring's Jackson library converts them to and from JSON automatically.

Class	Direction	JSON example
NewGameRequest	browser → server	{"playerName": "Giri", "difficulty": "HARD"}
PlayRequest	browser → server	{"move": "ROCK"}
GameStateResponse	server → browser	{"gameId": "…", "playerName": "Giri", "opponent": "Hard (learns your habits)", "lastRound": {…}, "wins": 1, "losses": 0, "draws": 2, "history": […]}

`GameStateResponse.from(id, game)` is a static factory method that reads the Game object and builds the response. Keeping this conversion out of the domain model means the model never has to know about JSON or HTTP.

4.6 service package – business logic

GameService.java is annotated with `@Service`, so Spring creates one instance and injects it into the controller. It stores all running games in a `ConcurrentHashMap<String, Game>` keyed by a random UUID.

- `createGame(name, difficulty)` – applies defaults (name "Player", difficulty EASY), builds a HumanPlayer and a ComputerPlayer with the right strategy, stores the Game and returns its id.
- `getGame(id)` – returns the game or throws GameNotFoundException.
- `play(id, move)` – validates the move and plays a round inside `synchronized (game)`, so two fast clicks can't corrupt the same game.
- `reset(id)` – resets one game.

GameNotFoundException.java – a custom RuntimeException annotated with `@ResponseStatus(HttpStatus.NOT_FOUND)`, so Spring automatically answers with HTTP 404 when an unknown game id is used.

4.7 controller package – the REST API

GameController.java is a `@RestController` mapped to `/api/games`. It receives the GameService through its constructor (**constructor dependency injection**) and contains no game logic – it only translates between HTTP and the service.

Method & URL	Body	What it does
POST /api/games	NewGameRequest	Creates a new game and returns its state.
GET /api/games/{id}	–	Returns the current state of a game.
POST /api/games/{id}/play	PlayRequest	Plays one round and returns the new state.
POST /api/games/{id}/reset	–	Clears score and history.

An `@ExceptionHandler` turns an `IllegalArgumentException` (e.g. a missing move) into an HTTP 400 response with a JSON error message.

4.8 Frontend – index.html and style.css

The UI is a single HTML file using **Vue 3** from a CDN. Vue keeps the page in sync with the data: when the `game` object changes, the score, result and history table update automatically.

- **data()** – the page's state: playerName, difficulty, the current game (null on the setup screen), a busy flag and an error message.
- **v-if / v-else** – shows the setup card when there is no game, otherwise the game card.
- **v-for** – draws the three move buttons and the history rows.
- **api(path, body)** – one helper that POSTs JSON with `fetch()`, stores the returned GameStateResponse in `this.game` and handles errors.
- **startGame / play / reset** – call the matching REST endpoints.
- **emoji() / message()** – turn enum names such as ROCK or WIN into emoji and friendly text.

```
async api(path, body) {
  const res = await fetch('/api/games' + path, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body || {})
  });
  this.game = await res.json();   // Vue re-renders automatically
}
```

style.css defines colour variables in `:root`, a centred card layout, a three-column score bar, large emoji move buttons with a hover lift, and green / red / orange colours for win / lose / draw.

4.9 Tests and build file

GameLogicTest.java (JUnit 5) has three tests:

- **rulesAreCorrect** – every win, loss and draw combination of Outcome.decide().
- **gameTracksScoreAndHistory** – uses an anonymous class strategy that always plays ROCK (a test double, made possible by the interface) and checks the score, history and reset.
- **frequencyStrategyCountersFavouriteMove** – after seeing ROCK twice, the Hard AI must play PAPER.

pom.xml – the Maven build file. It inherits from `spring-boot-starter-parent` (which manages library versions), adds `spring-boot-starter-web` and `spring-boot-starter-test`, and uses the Spring Boot Maven plugin so `mvn spring-boot:run` and `mvn package` work.

5. What happens when you click “Rock”

Following one click through the whole system shows how all the modules work together:

#	Where	What happens
1	index.html	The Rock button calls `play('ROCK')`.
2	index.html	`fetch()` sends POST /api/games/{id}/play with {"move":"ROCK"}.
3	Spring / Jackson	The JSON is converted into a `PlayRequest` record; "ROCK" becomes `Move.ROCK`.
4	GameController	`play()` calls `service.play(id, move)`.
5	GameService	Finds the Game in the map, locks it and calls `game.playRound(ROCK)`.
6	Game	Asks HumanPlayer and ComputerPlayer for their moves (polymorphic `chooseMove()`).
7	Strategy	ComputerPlayer asks its strategy: random (Easy) or counter your favourite move (Hard).
8	Outcome / Move	`Outcome.decide()` uses `Move.beats()` to get WIN, LOSE or DRAW.
9	Game / Scoreboard	A new Round is stored, the score is updated and the AI observes your move.
10	GameController	`GameStateResponse.from()` builds the reply; Spring converts it to JSON.
11	index.html	Vue stores the response in `this.game` and the screen updates instantly.

6. How to run the project

Requirements: **Java 17 or newer** and **Maven**.

```
cd rock-paper-scissors
mvn spring-boot:run          # start the server
# open http://localhost:8080 in your browser

mvn test                     # run the unit tests
mvn package                  # build target/rock-paper-scissors-1.0.0.jar
java -jar target/rock-paper-scissors-1.0.0.jar
```

In IntelliJ IDEA or Eclipse: open the folder (or pom.xml) as a Maven project and run `RockPaperScissorsApplication`.

7. Limitations and ideas for improvement

- Games are kept in memory, so they disappear when the server restarts – a database (e.g. Spring Data JPA + H2) could store them.
- The UI loads Vue from a CDN, so it needs internet access the first time.
- New AI levels: implement MoveStrategy (e.g. a pattern-based strategy that looks at your last two moves) and add a Difficulty constant – no other class needs to change.
- Game modes such as “best of 5”, or Rock-Paper-Scissors-Lizard-Spock by extending the Move enum.
- Multiplayer: a second HumanPlayer subclass driven by another browser.